Ministère de l'Enseignement
Supérieur et de la Recherche
Scientifique

Année Universitaire
2025/2026

EPI DIGITAL SCHOOL
Ecole Internationale Supérieure
Privée Polytechnique de Sousse

RAPPORT DE STAGE PROFESSIONNEL

Spécialité: .....

Intitulé

.....

Lieu du stage **Réalisé par** **Encadré par**

Dédicace

Remerciements

Résumé

Ce rapport présente la conception et la réalisation d'une application web interne dédiée à la gestion des demandes techniques. Le projet couvre l'enregistrement, la qualification, l'affectation, le traitement, la résolution et la clôture des demandes, avec une historisation des changements importants. L'architecture associe un frontend Angular, une API REST Spring Boot et une base PostgreSQL, selon une organisation modulaire séparant présentation, application, domaine et persistance. La sécurité repose sur l'authentification JWT, le contrôle d'accès fondé sur les rôles et des contrôles métier contextuels. Une assistance IA intégrée via Groq propose une catégorie et une priorité à partir du titre et de la description ; le Responsable technique conserve la décision finale. La validation s'appuie sur des tests automatisés backend et frontend, une intégration locale du parcours IA et une vérification fonctionnelle humaine. Le rapport distingue les résultats mesurés sur le périmètre V1 des limites, notamment le caractère exploratoire du benchmark IA et l'absence de portail client.

Mots-clés : génie logiciel ; API REST ; Angular ; Spring Boot ; JWT ; intelligence artificielle.

Abstract

This report presents the design and implementation of an internal web application for managing technical requests. The project covers request registration, qualification, assignment, processing, resolution and closure, with the history of significant changes. The architecture combines an Angular frontend, a Spring Boot REST API and a PostgreSQL database, using modular responsibilities for presentation, application, domain and persistence. Security relies on JWT authentication, role-based access control and contextual business checks. An AI assistance feature integrated through Groq proposes a category and a priority from the request title and description ; the Technical Manager remains responsible for the final decision. Validation combines automated backend and frontend tests, a local integration of the AI workflow and human functional verification. The report distinguishes measured results within the V1 scope from remaining limitations, including the exploratory nature of the AI benchmark and the absence of a customer portal.

Mots-clés

Sommaire

Introduction générale **1**

1 Contexte et cadrage du projet **2**
1.1 Cadre professionnel et informations disponibles 3
1.2 Contexte fonctionnel 3
1.3 Problématique et objectifs 3
1.4 Étude de l'existant et limites du périmètre 3
1.5 Acteurs et responsabilités 4
1.6 Démarche et organisation du travail 4
1.7 Synthèse du cadrage 4

2 Analyse et conception **5**
2.1 De la problématique aux exigences 6
2.2 Règles métier et cycle de vie 6
2.3 Architecture retenue 6
2.4 SOLID et patterns : analyse limitée aux preuves 7
2.5 Conception des données 7
2.6 API, sécurité et session Angular 7
2.7 Conception de l'assistance IA 8

3 Réalisation et validation **9**
3.1 Vue d'ensemble de la solution réalisée 10
3.2 Parcours complet d'une demande 10
3.3 Administration et contrôle des accès 13
3.4 Assistance IA et validation humaine 13
3.5 Tests et validation 14
3.6 Limites et perspectives 14

Conclusion générale **16**

Annexes **17**

Références **18**

Table des figures

1 Connexion à l'application 10
2 Création et qualification initiale d'une demande 11
3 Liste filtrable des demandes 11
4 Traitement d'une demande affectée 12
5 Clôture après examen de la résolution 12
6 Gestion des utilisateurs internes 13
7 Assistance IA lors de la qualification 14

Liste des tableaux

1 Synthèse des exigences principales 6

2 Niveaux de validation et portée 15

Introduction générale

Le stage porte sur la conception et la réalisation d'une application web interne destinée à organiser les demandes techniques reçues des clients. L'enjeu est de qualifier les demandes, attribuer les responsabilités, sécuriser les accès, suivre les transitions et conserver une trace des changements.

La problématique est la suivante : comment construire une solution simple à exploiter, maintenable et contrôlée pour accompagner tout le cycle de vie d'une demande, tout en conservant une décision humaine lorsque l'assistance IA est sollicitée ?

Le projet vise une architecture Angular, Spring Boot et PostgreSQL, structurée en modules et protégée par JWT et RBAC. Le rapport présente d'abord le contexte et le cadrage, puis l'analyse et la conception, avant de décrire la réalisation et la validation. Les informations administratives absentes restent signalées comme éléments à compléter.

Chapitre 1

Contexte et cadrage du projet

1.1 Cadre professionnel et informations disponibles

Le projet étudié concerne une application interne de gestion des demandes techniques. Les sources disponibles décrivent précisément le besoin logiciel et la solution réalisée, mais elles ne fournissent pas les informations administratives nécessaires pour présenter l'organisme d'accueil, son secteur d'activité, l'équipe ou l'encadrement. Ces éléments sont donc laissés à compléter : nom officiel de l'organisme, activité, service d'accueil, encadrants et dates du stage.

Cette absence ne doit pas être compensée par une description générique. Le rapport distingue le contexte logiciel, vérifiable dans le projet, du contexte professionnel, qui devra être renseigné à partir des informations communiquées par l'étudiant ou l'organisme d'accueil.

1.2 Contexte fonctionnel

Une demande technique naît lorsqu'un client rencontre un problème ou formule un besoin nécessitant l'intervention d'une équipe technique. La difficulté n'est pas limitée à la saisie initiale. Une demande doit être suffisamment décrite pour être comprise, qualifiée pour être priorisée, affectée à une personne responsable, puis suivie jusqu'à une résolution acceptée ou à une clôture justifiée.

Le projet vise donc à fournir un espace commun où les rôles, les décisions et l'état courant d'une demande restent visibles. La V1 se concentre sur les utilisateurs internes : le client est représenté comme une donnée métier, mais ne se connecte pas à l'application.

1.3 Problématique et objectifs

La problématique retenue est la suivante : comment construire une application interne capable de piloter le cycle de vie d'une demande technique, en conciliant simplicité d'utilisation, contrôle des droits, cohérence des données et traçabilité des décisions ?

Les objectifs sont d'enregistrer une demande, donner à chaque rôle une vision cohérente, formaliser les transitions, conserver l'historique, protéger l'API, proposer une assistance IA facultative et rendre le fonctionnement vérifiable par des tests et une validation humaine. L'objectif n'est pas de multiplier les fonctionnalités, mais de construire un parcours cohérent sur un périmètre V1 maîtrisé.

1.4 Étude de l'existant et limites du périmètre

Les sources du projet ne documentent pas un processus métier antérieur observé dans une entreprise. Il serait donc incorrect d'affirmer que l'application remplace un outil précis ou corrige

des délais mesurés. L'étude de l'existant est limitée aux besoins identifiés, aux rôles, aux étapes du cycle et aux exclusions de la V1.

Le périmètre comprend les demandes internes, les clients associés, la qualification, l'affectation, le traitement, la résolution, la clôture, le refus de résolution, l'annulation avec motif, l'historisation et l'administration des utilisateurs. Il exclut le portail client, le compte client, un module autonome de gestion de clients, les sous-catégories, le statut d'attente, les commentaires, la suppression fonctionnelle et la décision IA automatique.

Cette délimitation maîtrise le risque et évite de confondre une perspective future avec une fonction livrée.

1.5 Acteurs et responsabilités

Le Responsable technique enregistre et qualifie les demandes, organise l'affectation et examine les résolutions. L'Agent technique traite les demandes qui lui sont affectées, renseigne la solution et déclare la résolution. L'Administrateur gère les comptes internes et leurs rôles métier ; ce rôle administratif ne donne pas automatiquement accès aux opérations métier. Un utilisateur peut cumuler plusieurs rôles, ce qui explique la vérification de l'ensemble des rôles et du contexte de la demande.

1.6 Démarche et organisation du travail

La démarche suit une progression de génie logiciel : formalisation du besoin et du périmètre, modélisation des règles de cycle de vie, conception de l'architecture et des données, réalisation, puis validation. La validation distingue les tests automatisés, les vérifications d'intégration locale et l'observation humaine des parcours dans le navigateur.

1.7 Synthèse du cadrage

Le cadrage fait apparaître une tension centrale : l'application doit guider le processus sans empêcher les décisions métier. Les statuts, les rôles et l'historique structurent le fonctionnement ; l'assistance IA réduit l'effort de qualification mais reste une proposition. Le chapitre suivant montre comment cette tension a été traduite en exigences, modèles et choix d'architecture.

Chapitre 2

Analyse et conception

2.1 De la problématique aux exigences

Le besoin initial est exprimé en termes de suivi de demandes. Pour devenir développable, il est transformé en exigences observables : qui réalise l'action, sur quelle donnée, à quel moment et sous quelles conditions ? Les exigences fonctionnelles couvrent l'authentification, la gestion des demandes et clients, le workflow, l'administration et l'assistance IA. Les exigences non fonctionnelles ajoutent des contraintes de sécurité, cohérence transactionnelle, maintenabilité, ergonomie, accessibilité et traçabilité.

TABLE 1 – Synthèse des exigences principales

Domaine	**Attente principale**
Authentification	Identifier un utilisateur actif et transmettre un JWT aux requêtes protégées.
Demandes	Créer, consulter, filtrer, affecter, traiter, résoudre, clôturer ou annuler une demande.
Administration	Gérer les comptes, rôles métier et états actifs.
Sécurité	Appliquer RBAC et contrôles contextuels côté backend.
IA	Proposer une catégorie et une priorité sans application automatique.

2.2 Règles métier et cycle de vie

Une demande ne se résume pas à un statut modifiable librement. Chaque transition correspond à une action métier explicite. L'affectation associe une demande à un agent ; le traitement permet de renseigner les éléments nécessaires ; la résolution est déclarée par l'Agent puis examinée par le Responsable technique. Le refus renvoie la demande vers une reprise. L'annulation exige un motif et reste distincte de la clôture. Les changements métier et leur historisation sont atomiques.

Le diagramme d'activité répond à la question des transitions possibles depuis chaque état. Le diagramme de cas d'utilisation montre quel acteur demande quelle capacité. Les séquences validées détaillent les scénarios majeurs sans exposer le détail des contrôleurs et repositories ; le lecteur comprend d'abord le comportement, avant l'implémentation.

2.3 Architecture retenue

La solution combine Angular, une API REST Spring Boot et PostgreSQL. Le backend est un monolithe modulaire : les fonctionnalités partagent un déploiement, mais sont séparées autour

de ‘request‘, ‘identity / administration‘, ‘security‘ et ‘ai‘. Cette organisation répond au besoin de simplicité opérationnelle tout en préparant une évolution maîtrisée.

La séparation interne distingue présentation, application, domaine et persistance. Le Controller traduit HTTP vers un cas d'utilisation ; le Service orchestre ce cas et sa transaction ; le domaine porte les invariants ; le Repository encapsule la persistance ; les DTO stabilisent les échanges et séparent contrat API et entité JPA.

2.4 SOLID et patterns : analyse limitée aux preuves

Le principe de responsabilité unique est observable dans la séparation entre contrôleurs, services, objets métier et repositories. Cette organisation améliore la lisibilité, mais ne prouve pas une conformité absolue à SOLID. Le principe d'inversion des dépendances est visible dans ‘AnalyseIaPort‘ : le cœur applicatif dépend d'une abstraction et l'adaptateur Groq fournit l'intégration concrète.

Le projet n'utilise pas le State Pattern pour le cycle de vie. Les états sont portés par un enum et des opérations métier explicites, solution adaptée au nombre limité d'états de la V1. Cette décision pourra être réévaluée si le workflow devient beaucoup plus complexe.

2.5 Conception des données

Les entités persistées sont ‘Utilisateur‘, ‘Client‘, ‘DemandeTechnique‘ et ‘HistoriqueDemande‘. La référence métier est générée côté serveur, unique et immuable ; l'email utilisateur est unique et normalisé. JPA et Hibernate assurent le mapping objet-relationnel, Spring Data JPA fournit les repositories et Flyway applique les migrations SQL versionnées. Les instants utilisent ‘Instant‘ côté Java et ‘timestamptz‘ côté PostgreSQL (PostgreSQL Global Development Group 2026).

2.6 API, sécurité et session Angular

L'API REST expose des opérations orientées métier et retourne des DTO. Le backend est l'autorité pour l'identité, l'état actif, les rôles courants et les permissions. Le contrôle d'accès fondé sur les rôles (RBAC) est complété par des contrôles contextuels, notamment pour l'affectation et l'auto-désactivation.

Le JSON Web Token (JWT) est un format compact de claims standardisé par l'IETF (Jones, Bradley et Sakimura 2015). Angular conserve la session en mémoire et dans ‘sessionStorage‘ sous ‘safyron.session.v1‘. Il effectue des contrôles locaux de structure et d'expiration, sans remplacer la validation backend de la signature et des droits. Les mots de passe sont traités par ‘PasswordEncoder‘ et la configuration sensible est externalisée (Spring 2026b ; Spring 2026a).

2.7 Conception de l'assistance IA

L'IA réduit l'effort de qualification, mais ne prend pas la décision. L'entrée est limitée au titre et à la description ; la sortie propose une catégorie et une priorité. ‘AnalyseIaPort‘ isole le cœur applicatif de l'adaptateur Groq. La proposition est temporaire, modifiable et appliquée après un clic explicite. En cas d'indisponibilité, la saisie manuelle reste possible.

Chapitre 3

Réalisation et validation

3.1 Vue d'ensemble de la solution réalisée

Angular fournit l'application web et les parcours utilisateur. Spring Boot expose l'API et applique les règles métier. PostgreSQL conserve les données et l'historique. Cette répartition permet de faire évoluer l'interface sans déplacer les invariants métier dans le navigateur.

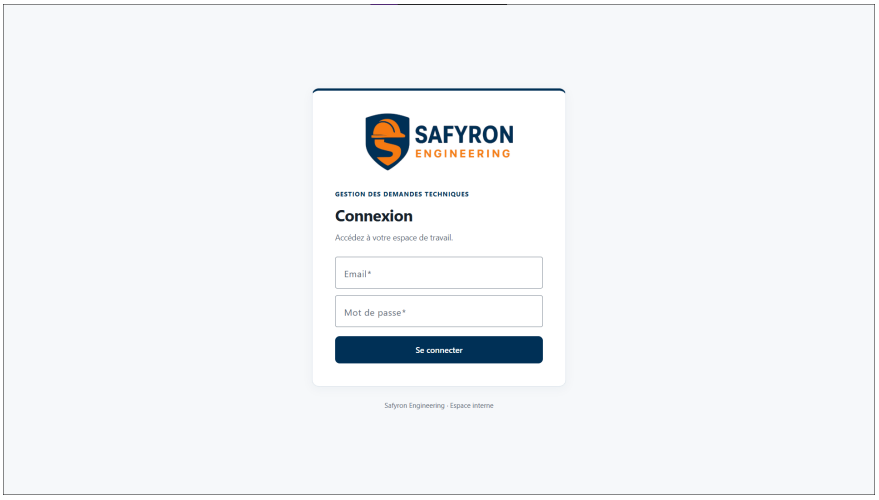


FIGURE 1 – Connexion à l'application

Après authentification, le frontend conserve le token et le snapshot minimal utile à l'orientation de l'interface. Les autorisations effectives restent décidées par l'API.

3.2 Parcours complet d'une demande

Le parcours commence par la création et la qualification. Le Responsable technique renseigne un titre, une description, un client, une catégorie et une priorité. Il peut demander une suggestion IA, mais garde la possibilité de modifier les valeurs avant l'enregistrement.

La liste permet ensuite de retrouver les demandes par filtres, tri et pagination côté serveur. Après affectation, l'Agent traite la demande, renseigne la solution et déclare la résolution. Le Responsable examine cette proposition : il clôture la demande ou la refuse pour demander une reprise.

Ces écrans illustrent une propriété importante : l'interface présente des actions dépendantes du rôle et du statut, mais cette présentation ne remplace pas le contrôle côté serveur.

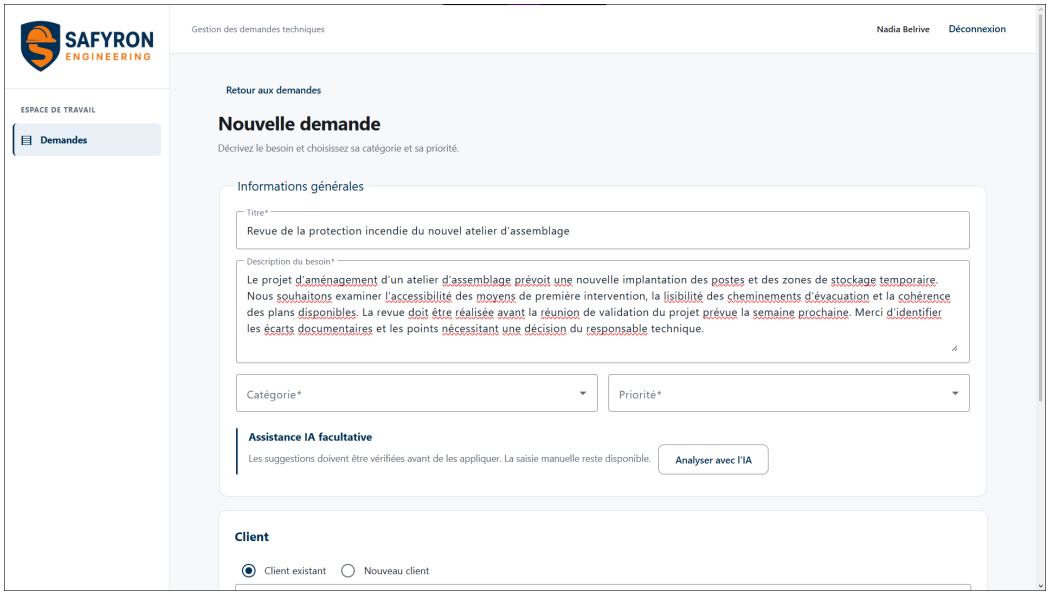


FIGURE 2 – Création et qualification initiale d'une demande

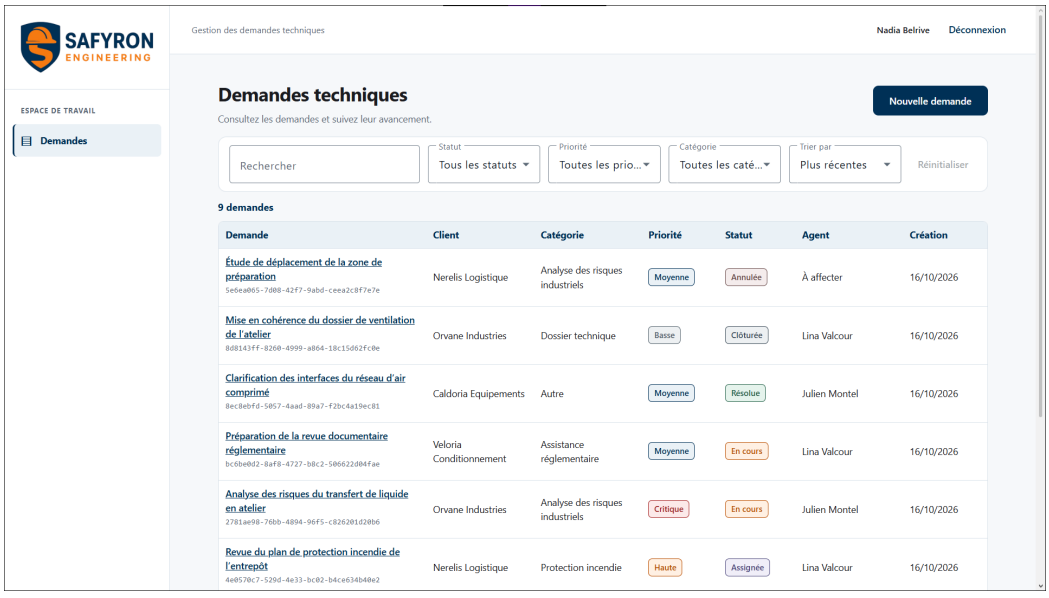


FIGURE 3 – Liste filtrable des demandes

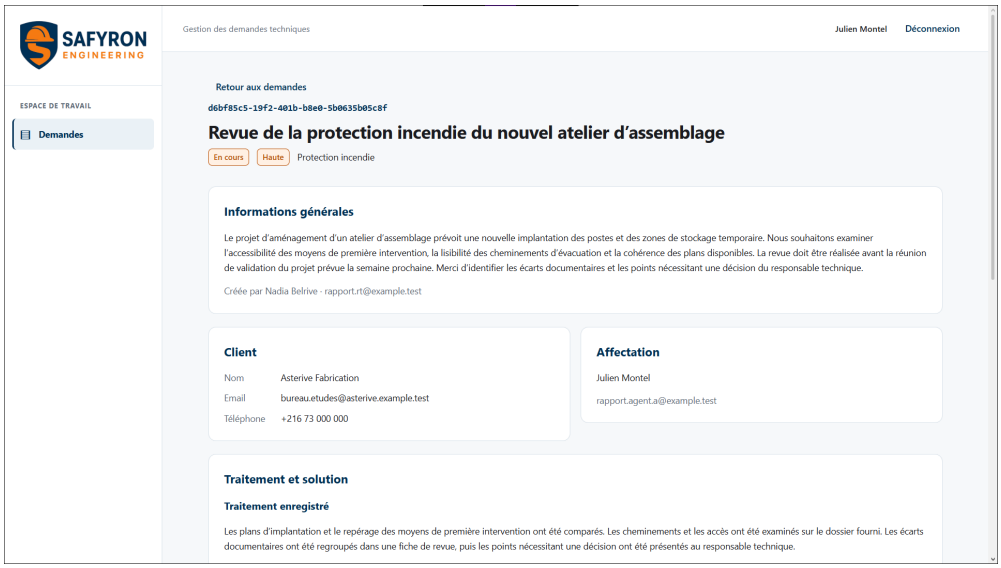


FIGURE 4 – Traitement d'une demande affectée

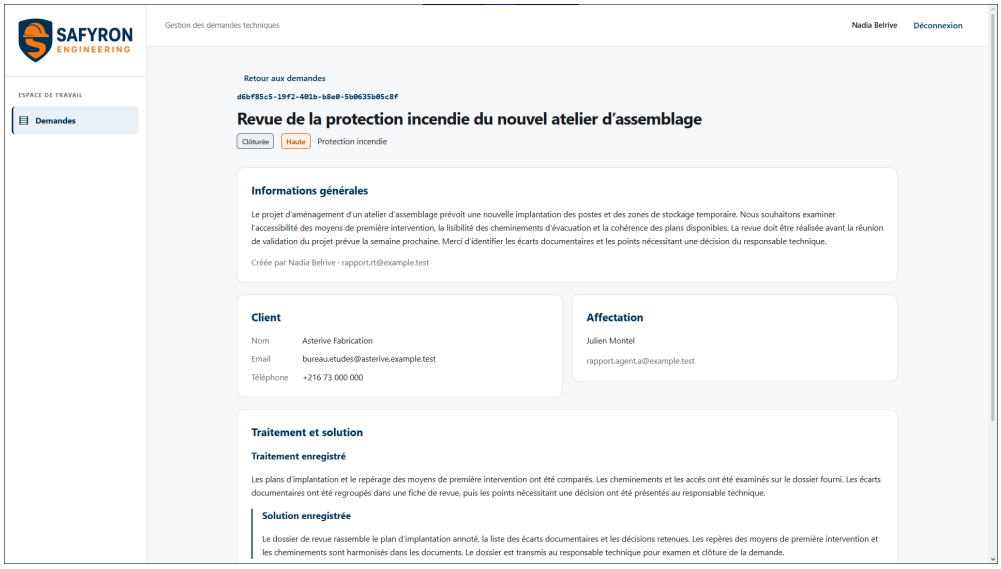


FIGURE 5 – Clôture après examen de la résolution

3.3 Administration et contrôle des accès

L'administration permet de créer et modifier les comptes, de gérer leur état actif et de remplacer leurs rôles métier. L'auto-désactivation du compte courant est refusée par l'API avec une réponse 409 et l'interface conserve un retour compréhensible.

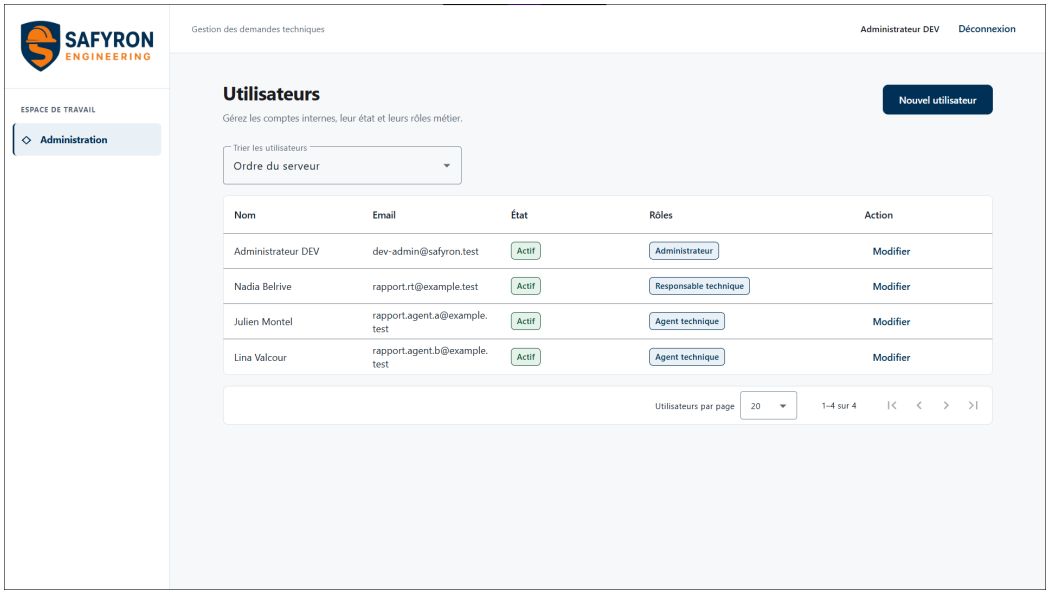


FIGURE 6 – Gestion des utilisateurs internes

Le backend relit l'utilisateur et ses rôles à chaque requête protégée. Un rôle supprimé ou un compte désactivé ne conserve donc pas ses droits simplement parce qu'un snapshot ancien existe dans le navigateur.

3.4 Assistance IA et validation humaine

L'analyse IA est déclenchée depuis le formulaire de nouvelle demande. Le frontend envoie uniquement le titre et la description à l'endpoint backend. Le serveur appelle l'adaptateur Groq via ‘AnalyseIaPort‘, contrôle la réponse puis renvoie une proposition temporaire. La catégorie et la priorité ne sont jamais appliquées automatiquement.

Une modification du titre ou de la description invalide la suggestion. Lorsque la clé Groq est absente, l'API renvoie une indisponibilité contrôlée et le formulaire reste utilisable manuellement. Le benchmark exploratoire porte sur 24 exemples : sorties valides dans les 24 cas, catégorie correcte dans 23 cas et priorité dans 22 cas. Ces valeurs ne permettent pas d'annoncer une performance générale ni une supériorité universelle d'un fournisseur.

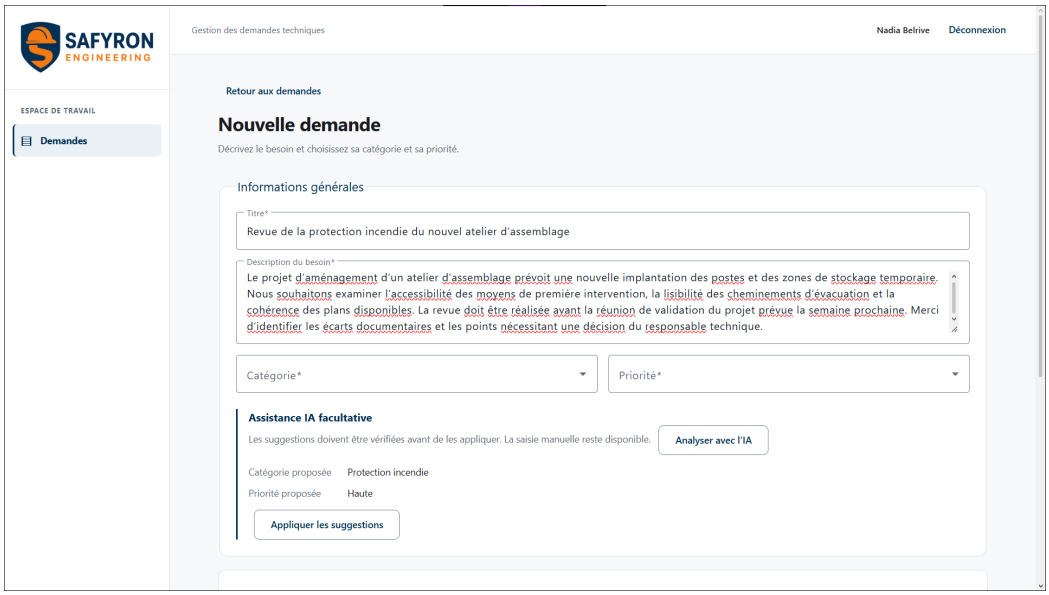


FIGURE 7 – Assistance IA lors de la qualification

3.5 Tests et validation

La validation backend atteint 270 tests réussis avec ‘clean test‘ et ‘clean verify‘, sans échec, erreur ou test ignoré. Les tests IA hors ligne utilisent H2 et un stub HTTP local ; ils ne constituent pas un appel Groq. Le frontend a été vérifié par ‘npm ci‘, ‘npm run build‘ et ‘npm test ––watch=false‘, avec 184 tests réussis dans 15 fichiers pour la clôture IA.

La validation fonctionnelle humaine a couvert la connexion, la liste, la création, la sélection client, les transitions de demande, l’administration, l’auto-désactivation et l’assistance IA réelle en local. Une revue responsive et accessibilité a également été effectuée sur plusieurs tailles. Ces observations ne remplacent pas une campagne de test de charge ou une validation de production.

3.6 Limites et perspectives

Le rapport ne prétend pas mesurer la performance en production, la résistance à la charge ou la qualité IA sur un corpus représentatif du long terme. Le périmètre reste interne et le client n’interagit pas directement avec la plateforme. La session navigateur utilise ‘sessionStorage‘, ce qui permet une restauration après rechargement mais conserve les risques propres à un stockage accessible au JavaScript du navigateur.

Les perspectives cohérentes avec la V1 sont l’élargissement du corpus d’évaluation IA, l’ajout de mesures répétées et l’amélioration de l’observabilité. Toute extension du workflow, du

TABLE 2 – Niveaux de validation et portée

Validation	**Ce qu'elle permet d'affirmer**
Tests backend	Les comportements couverts par 270 tests réussissent dans l'environnement vérifié.
Tests frontend	La construction et les scénarios automatisés couverts réussissent.
Intégration locale IA	Le parcours Angular vers Spring Boot et Groq a fonctionné dans les conditions observées.
Validation humaine	Les parcours et l'ergonomie vérifiés ne présentent pas de blocage constaté dans le périmètre revu.
Benchmark IA	Résultats exploratoires sur 24 exemples, non généralisables.

portail client ou du déploiement devra être précédée d'une analyse de besoin et d'une validation dédiée.

Conclusion générale

Le stage a conduit à une application V1 couvrant l'enregistrement, la qualification, l'affectation, le traitement et le suivi des demandes techniques. La séparation entre interface Angular, API Spring Boot et persistance PostgreSQL fournit une base lisible et évolutive. La sécurité combine authentification JWT, rôles métier et contrôles contextuels.

L'assistance IA facultative via Groq propose une catégorie et une priorité, mais la décision finale reste modifiable et validée par le Responsable technique. Les tests automatisés et les validations humaines montrent que le périmètre V1 est fonctionnel dans les conditions vérifiées.

Les limites concernent le périmètre interne, l'absence de portail client et le caractère exploratoire du benchmark IA. Des travaux ultérieurs pourraient élargir le corpus d'évaluation, renforcer l'observabilité et étudier les besoins de déploiement, sous réserve de validations supplémentaires.

Annexes

Références

Jones, Michael, John Bradley et Nat Sakimura (mai 2015). *JSON Web Token (JWT)*. url : https://www.rfc-editor.org/rfc/rfc7519.html (visité le 11/10/2026).

PostgreSQL Global Development Group (2026). *Date/Time Types*. url : https://www.postgresql.org/docs/current/datatype-datetime.html (visité le 11/10/2026).

Spring (2026a). *Externalize the Configuration of SpringApplication*. url : https://docs.spring.io/spring-boot/how-to/properties-and-configuration.html (visité le 11/10/2026).

— (2026b). *Password Storage*. url : https://docs.spring.io/spring-security/reference/7.0/features/authentication/password-storage.html (visité le 11/10/2026).

Table des matières détaillée

Introduction générale **1**

1 Contexte et cadrage du projet **2**
1.1 Cadre professionnel et informations disponibles 3
1.2 Contexte fonctionnel 3
1.3 Problématique et objectifs 3
1.4 Étude de l'existant et limites du périmètre 3
1.5 Acteurs et responsabilités 4
1.6 Démarche et organisation du travail 4
1.7 Synthèse du cadrage 4

2 Analyse et conception **5**
2.1 De la problématique aux exigences 6
2.2 Règles métier et cycle de vie 6
2.3 Architecture retenue 6
2.4 SOLID et patterns : analyse limitée aux preuves 7
2.5 Conception des données 7
2.6 API, sécurité et session Angular 7
2.7 Conception de l'assistance IA 8

3 Réalisation et validation **9**
3.1 Vue d'ensemble de la solution réalisée 10
3.2 Parcours complet d'une demande 10
3.3 Administration et contrôle des accès 13
3.4 Assistance IA et validation humaine 13
3.5 Tests et validation 14
3.6 Limites et perspectives 14

Conclusion générale **16**

Annexes **17**

Références **18**